

Exercise 6: Plotting and Coordinate Spaces

Note: Before you start this exercise, if you already have Octave running and some variables defined, remove them with the “clear” command. Make a folder for today’s work in your Octave working directory, with a dated label in YYMMDD format. Remember to make notes in your research notebook as well!

1. Plotting in Octave

Now that we you comfortable organizing large amounts of data into multidimensional arrays, you need a way to look at these big data sets and quickly determine their properties. A great way to do this is by creating a picture of your data set. You can make a simple 2D plot using the `plot( )` command. We will explore the properties of this command using the code below, type it up as an Octave script and run it.

```
%file created by XXX on YYMMDD
%tutorial exercise 6.1

clear;

% define some variables
horz_variable = -10:1:10;
vert_variable1 = 3 .* horz_variable - 7;
vert_variable2 = -5 .* horz_variable + 10;

% build a figure
fig1 = figure
hold on;
plot(horz_variable, vert_variable1, '-b');
plot(horz_variable, vert_variable2, '+r');
xlabel('Horizontal Axis Quantity (unit)');
ylabel('Vertical Axis Quantity (unit)');
title('Figure Title');
hold off;

% write the figure to a file
print(fig1, 'testplot.png', '-dpng');
close;
```

The first block of code, lines 6-9, defines some variables for us to plot. We need an independent variable for our 2D plot, which we create as `horz_variable` in line 7. The code -

`10:1:10` instructs Octave to create a 1D array containing values starting at -10, increasing by 1 for each successive element in the array, and finishing at a value of 10.

The second block of code, lines 11-19, instructs Octave to build a figure. In line 12 we create a variable *fig1* and set it as a “figure handle”. A figure handle is a lot like the *file_ID* variable from Exercise 5, it holds the location and status of our figure. If we wind up creating multiple figures, we can use *fig1* to reference this specific figure. Line 13 turns on a figure property called *hold*. Octave’s default setting is to discard the results of any previous plotting commands when running a new plotting command. The *hold on* command tells Octave not to do this, but instead to keep all plots and show them on the same set of axes. Octave will keep doing this until we declare a new figure or give the *hold off* command.

Lines 14 and 15 plot two lines. The *plot()* command can take a lot of different styles of input, you can explore the options in chapter 15.2.1 of the online Octave manual (https://www.gnu.org/software/octave/doc/interpreter/Two_002dDimensional-Plots.html). The format shown is a common one for 2D plots, directing Octave to use the first array as the independent variable and the second array as the dependent variable. The third input on the command in line 14, ‘-b’, tells Octave to plot the data as a blue line. In line 15 the ‘+r’ command tells Octave to plot that data using red + shaped markers. Octave supports a lot of line and marker formats, you can explore them more fully in the online manual. Whenever you plot multiple data series, make them visually distinct. The choice between markers and lines is not usually critical, although it is most correct to use markers for discrete data points and lines for continuous functions. Lines 16-18 display text on the figure. Any figure you create should have axis labels with units. Figure titles are also often useful, especially if you are creating and/or reviewing lots of figures at once. Line 19 turns off the *hold* property for the figure.

The third block of code saves your figure to a file using the *print* command. The *print* command also takes several forms of input, you can get full details in the online manual chapter 15.2.1 (<https://www.gnu.org/software/octave/doc/interpreter/Printing-and-Saving-Plots.html>). In the format shown, the first input *fig1* is the figure handle for the figure to be saved. The second input is the file name as a text string. The third input is the desired file format as a text string, preceded by -d. The .png format is a good image format for inserting files into documents, because it is designed to rescale without distorting. Other useful formats include .pdf, which is

widely readable and generally creates small files, and .eps, which creates line-graphic files that can be edited in graphic design software such as Adobe Illustrator.

2. Vectors and Coordinate Spaces

Now that we can create a 2D plot, we can start thinking about vectors in a more visual way. Let's say we have a vector in a 2D coordinate space that extends two units along the horizontal axis and three units along the vertical axis. We can specify this vector as a two element array, and then look at it in a plot:

```
%file created by XXX on YYMMDD  
%tutorial exercise 6.2  
  
clear;  
  
vector_xy = [2 3];  
  
fig1 = figure  
plot(vector_xy(1),vector_xy(2),'*');  
xlim([0 5]);  
ylim([0 5]);  
xlabel('x Axis Coordinate (unitless)');  
ylabel('y Axis Coordinate (unitless)');  
title('2D vector plot');
```

The *xlim* and *ylim* functions set the limits on the figure axes. You can see that our two coordinates have specified position in our two coordinate x,y space. We can think of this position as defining a vector that starts at the origin and extends to this point.

We can specify the same position in a different two coordinate space, where one coordinate is the length of the vector, r , and the second coordinate is the angle of rotation up from the horizontal axis, θ . This new vector can be pictured in the same way as the first one, starting at the origin and extending to the point specified. The r coordinate can have any value, while θ is allowed to have values between 0 and 2π (in radians). We have to set limits on θ to avoid redundancy. For example, if we let $(1, 2\pi)$ and $(1, 4\pi)$ both exist in r, θ space, they both specify the same point. This would cause logical problems when we try to do math using the coordinate values.

The plot command always works with x,y coordinates, so if we want to define these our vector in terms of r and θ we must convert to x and y before plotting. To explore this, we can change our code to calculate r and θ from our original x and y, then convert back for plotting.

```
%file created by XXX on YYMMDD
%tutorial exercise 6.2

clear;

vector_xy = [2 3];

% calculate r and theta from x,y coords
r_1 = sqrt(vector_xy(1)^2+vector_xy(2)^2); %using math formula
r_2 = norm(vector_xy); % using library function
r = r_1; %either r_1 or r_2 is fine here, both should be the same
theta = atan2(vector_xy(2),vector_xy(1)); %in radians

%calculate x and y from r and theta
x = r * cos(theta);
y = r * sin(theta);

%make a figure to compare
fig1 = figure
hold on;
plot(vector_xy(1),vector_xy(2),'*');
plot(x,y,'d','markersize',10);
xlim([0 5]);
ylim([0 5]);
xlabel('x Axis Coordinate (unitless)');
ylabel('y Axis Coordinate (unitless)');
title('2D vector plot');
hold off;
```

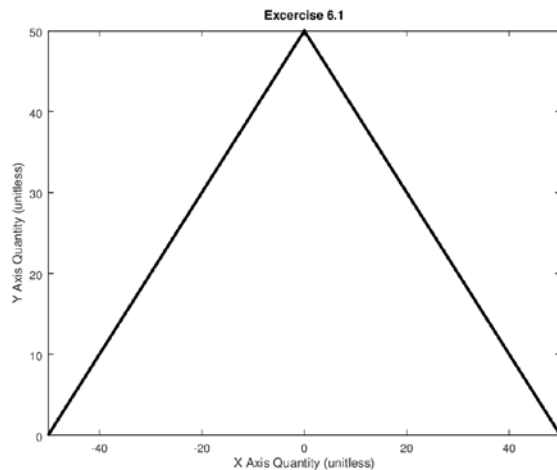
There is a lot going on here. The first new block of code, lines 8-12, takes our x,y coordinates and uses them to calculate r and θ coordinates using known mathematical formulas. The second new block, lines 14-16, takes the r and theta values and uses them to recalculate x,y coordinates. Note that the new x,y coordinates are stored in new variables, so that we can check them against the original variable `vector_xy` and make sure they come back the same. Changes to the last block of code plot both of our x,y coordinate sets to confirm that they are the same.

This r, θ coordinate system is called “polar coordinates” or “circular coordinates”. You can sketch out some triangles and do some trigonometry to prove the r, θ to x,y and x,y to r, θ

conversion formulas to yourself. There is an analogous coordinate system for three dimensional x,y,z space called “spherical polar coordinates”, which you will deal with in the exercises.

3. Exercises (Complete before moving on, use more than one session if needed)

1. Create and save a figure that shows a triangular function that starts at the x,y coordinates (-50,0), increases in a straight line to a maximum value of (0,50), and then decreases to a value of (50,0). Apply your knowledge of control statements to create arrays holding values for independent



and dependent variables, then plot your function. Set the axis limits to minimize whitespace. Plot the function as a black line with linewidth 2. Include figure title and axis labels. Your output file should look like the one shown above.

2. Create a program analogous to the one from section 6.2 of this exercise, but for the 3D spherical polar coordinates. A good command for 3D plotting in Octave is `plot3()`, see online manual section 15.2.2 for details. The formulas for converting from x,y,z coordinates to r,θ,φ are:

$$x = r \sin \theta \cos \phi$$

$$y = r \sin \theta \sin \phi$$

$$z = r \cos \theta$$

$$r = (x^2 + y^2 + z^2)^{\frac{1}{2}}$$

$$\cos \theta = \frac{z}{(x^2 + y^2 + z^2)^{\frac{1}{2}}}$$

$$\tan \phi = \frac{y}{x}$$

If you use outside reference material, be aware that physics and math use different definitions of the variables θ and φ. This is annoying and confusing, but that’s the way it is. We use the physics convention in our group, which defines r as the length of the vector, θ as the angle down from the z axis, and φ as the angle rotating around from

the x axis. The math convention flips the definitions of θ and ϕ . Also, note that θ and ϕ have limits to avoid redundancy, just like our θ coordinate from the 2D polar coordinate system. The coordinate θ is allowed to have values ranging between 0 and π , while the coordinate ϕ ranges from 0 to 2π .